

KKP STORES

Production Deployment Guide — Ubuntu on Amazon EC2

A full step-by-step playbook to deploy the KKP STORES fashion ecommerce stack (React + FastAPI + MongoDB) to a single EC2 Ubuntu instance, with Nginx, SSL, supervisor, and systemd. Estimated time end-to-end: **~45–60 minutes**.

What you'll have at the end

- A live HTTPS website at **<https://yourdomain.com>**
- An admin panel at **<https://yourdomain.com/admin>**
- MongoDB running locally with persistent storage
- Nginx reverse-proxying **/api** → FastAPI on :8001
- Frontend static build served by Nginx
- Auto-restart on crash + auto-renewed Let's Encrypt certs

Prerequisites

- An AWS account with billing enabled
- A registered domain (Route 53, GoDaddy, Namecheap, etc.)
- SSH client (Terminal on macOS/Linux, or PuTTY on Windows)
- Your KKP STORES source code (git repository or zip)

1. Launch the EC2 instance

Log in to the AWS Console → EC2 → **Launch Instances**.

Recommended configuration

Setting	Value
Name	kkp-stores-prod
AMI	Ubuntu Server 24.04 LTS (or 22.04 LTS) — 64-bit (x86)
Instance type	t3.small (2 vCPU, 2 GB RAM) — t3.medium for >100 SKUs
Key pair	Create new → kkp-stores-key.pem — download and keep safe
Network settings	Allow SSH (22), HTTP (80), HTTPS (443) from anywhere
Storage	30 GB gp3 — enough for OS + MongoDB + image cache

Click **Launch instance**. After ~1 minute, note the **Public IPv4 address** from the EC2 console — e.g. 13.232.xxx.xxx.

2. Connect via SSH

On your local machine open a terminal and run:

```
chmod 400 ~/Downloads/kkp-stores-key.pem
ssh -i ~/Downloads/kkp-stores-key.pem ubuntu@<EC2_PUBLIC_IP>
```

If you see *The authenticity of host... can't be established*, type **yes** to continue.

3. Update Ubuntu & install base packages

```
sudo apt update && sudo apt upgrade -y
sudo apt install -y curl git build-essential ufw nginx supervisor \
  software-properties-common ca-certificates gnupg lsb-release
```

4. Install Node.js 20 (frontend build)

```
curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash -
sudo apt install -y nodejs
node -v && npm -v
sudo npm install -g yarn pm2
```

5. Install Python 3.11 + pip + venv

```
sudo add-apt-repository ppa:deadsnakes/ppa -y
sudo apt update
sudo apt install -y python3.11 python3.11-venv python3.11-dev python3-pip
python3.11 --version
```

6. Install MongoDB 7

```
curl -fsSL https://www.mongodb.org/static/pgp/server-7.0.asc | \
```

```
sudo gpg -o /usr/share/keyrings/mongodb-server-7.0.gpg --dearmor  
  
echo "deb [signed-by=/usr/share/keyrings/mongodb-server-7.0.gpg] \  
https://repo.mongodb.org/apt/ubuntu jammy/mongodb-org/7.0 multiverse" | \  
sudo tee /etc/apt/sources.list.d/mongodb-org-7.0.list  
  
sudo apt update && sudo apt install -y mongodb-org  
sudo systemctl enable --now mongod  
sudo systemctl status mongod --no-pager | head -5
```

Production tip: bind MongoDB to 127.0.0.1 only and enable auth. Edit **/etc/mongod.conf** — set `bindIp: 127.0.0.1`, then create an admin user and a `kkp_stores` user.

7. Clone or upload the KKP STORES code

Option A — clone from GitHub:

```
cd /home/ubuntu
git clone https://github.com/<your-org>/kkp-stores.git app
cd app
```

Option B — upload from your laptop:

```
# Run on your LOCAL machine
scp -i ~/Downloads/kkp-stores-key.pem -r ./app/ \
  ubuntu@<EC2_PUBLIC_IP>:/home/ubuntu/
```

8. Configure backend environment

```
cd /home/ubuntu/app/backend
cp .env.example .env # or create one
nano .env
```

Paste the following (replace placeholders):

```
MONGO_URL="mongodb://127.0.0.1:27017"
DB_NAME="kkp_stores_prod"
CORS_ORIGINS="https://yourdomain.com"
JWT_SECRET="<run: openssl rand -hex 32>"
ADMIN_EMAIL="admin@yourdomain.com"
ADMIN_PASSWORD="<a strong 12+ char password>"
EMERGENT_AUTH_URL="https://demobackend.emergentagent.com/auth/v1/env/oauth/session-data"
```

9. Install backend dependencies

```
cd /home/ubuntu/app/backend
python3.11 -m venv venv
source venv/bin/activate
pip install --upgrade pip
pip install -r requirements.txt
pip install uvicorn gunicorn # production server
```

Smoke test the API:

```
uvicorn server:app --host 0.0.0.0 --port 8001
# In another SSH session:
curl http://127.0.0.1:8001/api/products?limit=1
```

Stop with Ctrl+C once you see a JSON response.

10. Build the React frontend

```
cd /home/ubuntu/app/frontend
cp .env.example .env
nano .env
# Set: REACT_APP_BACKEND_URL=https://yourdomain.com

yarn install --frozen-lockfile
yarn build
# Output: /home/ubuntu/app/frontend/build/
```

11. Run backend as a systemd service

```
sudo nano /etc/systemd/system/kkp-backend.service
```

Paste:

```
[Unit]
Description=KKP STORES FastAPI backend
After=network.target mongod.service
Requires=mongod.service

[Service]
Type=simple
User=ubuntu
Group=ubuntu
WorkingDirectory=/home/ubuntu/app/backend
EnvironmentFile=/home/ubuntu/app/backend/.env
ExecStart=/home/ubuntu/app/backend/venv/bin/gunicorn \
    -w 4 -k uvicorn.workers.UvicornWorker \
    -b 127.0.0.1:8001 server:app
Restart=always
RestartSec=5
StandardOutput=append:/var/log/kkp-backend.log
StandardError=append:/var/log/kkp-backend.log

[Install]
WantedBy=multi-user.target

sudo systemctl daemon-reload
sudo systemctl enable --now kkp-backend
sudo systemctl status kkp-backend --no-pager | head -10
# Tail logs: sudo tail -f /var/log/kkp-backend.log
```

12. Configure Nginx reverse proxy

```
sudo nano /etc/nginx/sites-available/kkp-stores
```

Paste:

```
server {
    listen 80;
    server_name yourdomain.com www.yourdomain.com;

    client_max_body_size 25M;

    # React build
    root /home/ubuntu/app/frontend/build;
    index index.html;

    # FastAPI under /api
    location /api/ {
        proxy_pass http://127.0.0.1:8001;
        proxy_http_version 1.1;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }

    # SPA fallback
    location / {
```

```
        try_files $uri /index.html;
    }

    # Cache static assets aggressively
    location ~* \.(?:js|css|woff2?|png|jpg|jpeg|gif|svg|webp)$ {
        expires 30d;
        access_log off;
        add_header Cache-Control "public, immutable";
    }
}

sudo ln -s /etc/nginx/sites-available/kkp-stores \
    /etc/nginx/sites-enabled/kkp-stores
sudo rm /etc/nginx/sites-enabled/default
sudo nginx -t && sudo systemctl reload nginx
```

13. Point DNS to the EC2 IP

In your DNS provider (Route 53 / Namecheap / GoDaddy), create:

Type	Name	Value	TTL
A	@	<EC2_PUBLIC_IP>	300
A	www	<EC2_PUBLIC_IP>	300

Verify after ~5 minutes: **dig yourdomain.com +short** should return the EC2 IP.

14. Install free SSL with Let's Encrypt

```
sudo apt install -y certbot python3-certbot-nginx
sudo certbot --nginx -d yourdomain.com -d www.yourdomain.com \
  --non-interactive --agree-tos -m admin@yourdomain.com
```

Auto-renew is installed as a systemd timer:
sudo systemctl list-timers | grep certbot

After this step your site is live on **https://yourdomain.com** with auto-renewing certificates.

15. UFW firewall (recommended)

```
sudo ufw allow OpenSSH
sudo ufw allow 'Nginx Full'
sudo ufw --force enable
sudo ufw status
```

16. Smoke test the live site

```
# From your laptop:
curl -I https://yourdomain.com
curl https://yourdomain.com/api/products?limit=1 | head -c 400
curl -X POST https://yourdomain.com/api/auth/login \
  -H 'Content-Type: application/json' \
  -d '{"email":"admin@yourdomain.com","password":"<your-pw>"}'
```

Open **https://yourdomain.com** in a browser → you should see the Tata-CLiQ-style KKP STORES homepage with the South-Indian hero. Sign in with the admin credentials from step 8 to access **/admin**.

17. Maintenance & day-2 operations

Tail logs

```
# Backend
sudo journalctl -u kkp-backend -f
sudo tail -f /var/log/kkp-backend.log
# Nginx access/error
sudo tail -f /var/log/nginx/access.log /var/log/nginx/error.log
# MongoDB
sudo tail -f /var/log/mongodb/mongod.log
```

Deploy an update

```
cd /home/ubuntu/app
git pull origin main
```

```
# Backend deps changed?
source backend/venv/bin/activate
pip install -r backend/requirements.txt
sudo systemctl restart kkp-backend
```

```
# Frontend changed?
cd frontend && yarn install && yarn build
sudo systemctl reload nginx
```

Backup MongoDB nightly (cron)

```
sudo crontab -e
# Add:
0 3 * * * mongodump --db kkp_stores_prod \
  --out /home/ubuntu/backups/$(date +%F) \
  && find /home/ubuntu/backups -type d -mtime +14 -exec rm -rf {} +
```

Rotate JWT secret

```
openssl rand -hex 32 # paste into /home/ubuntu/app/backend/.env
sudo systemctl restart kkp-backend
# All existing sessions become invalid – users sign in again.
```

18. Troubleshooting

Symptom	Likely cause	Fix
502 Bad Gateway from Nginx	FastAPI service down	<code>sudo systemctl status kkp-backend</code> ; <code>sudo journalctl -u kkp-backend -n</code>
CORS error in browser console	CORS_ORIGINS in .env doesn't match frontend	Edit backend/.env, restart kkp-backend
MongoDB connection refused	mongod not running or bound elsewhere	<code>sudo systemctl status mongod</code> ; check /etc/mongod.conf bindIp
Login works, /api/auth/me 401	Frontend not sending cookies	Verify REACT_APP_BACKEND_URL matches site domain; rebuild + deploy
Site loads on http but not https	Certbot didn't run / DNS not propagated	Run <code>sudo certbot --nginx</code> on yourdomain.com; rerun <code>sudo certbot --nginx</code>
Frontend hot-reload trying to connect with dev mode	Built with dev mode	Always deploy `yarn build` output, never `yarn start`

19. Hardening checklist (production)

- ✓ Change the seeded admin password immediately after first login.
- ✓ Set MongoDB bindIp to 127.0.0.1 and enable auth in /etc/mongod.conf.
- ✓ Replace allow_origins=["*"] in server.py CORS with your domain only.
- ✓ Disable SSH password login, only allow key auth (already default on Ubuntu AMI).
- ✓ Restrict SSH source IP in the EC2 Security Group (your office IP).
- ✓ Enable AWS CloudWatch alarms on CPU and disk.
- ✓ Snapshot the EBS volume weekly via Lifecycle Manager.
- ✓ If you grow >5k orders/month, move MongoDB to Atlas M10 and remove local mongod.

That's it. Your KKP STORES is now live, secured with HTTPS, auto-restarting on crash, and backing up nightly. For revisions or scaling beyond a single EC2 instance (ALB + multiple FastAPI nodes, Redis cache, CDN for /static), reach out and we can extend this playbook.